

Unit-4 Client-server interaction using socket.io

Socket.io:

Socket.IO is a library in Node.js that enables real-time, bidirectional, and event-based communication between web clients and servers. It allows you to implement real-time features like chat applications, live feeds, online multiplayer games, and collaboration tools where updates need to be pushed to the clients without them having to refresh or request updates manually.

Key Concepts:

1. **Real-time communication:** Traditional HTTP requests are one-way (client to server), whereas Socket.IO allows both the server and the client to communicate with each other in real-time.
2. **Bidirectional:** Messages or data can be sent in both directions (from client to server and server to client) over a persistent connection.
3. **Event-driven:** Socket.IO uses custom events. You can listen for and emit events for specific types of messages, allowing modular and structured communication.
4. **WebSocket and fallback:** Socket.IO primarily uses WebSockets for communication but falls back to HTTP long-polling or other mechanisms if WebSockets aren't supported by the client or server.

Why Use Socket.IO?

- **Real-time updates:** Ideal for applications that require live data, like chat apps, live notifications, stock tickers, or multiplayer games.
- **Low latency:** Because the connection stays open, latency is minimized compared to traditional HTTP requests.
- **Cross-browser support:** Socket.IO handles browser differences, falling back to other transports when WebSocket is unavailable.
- **Simple API:** Easy to use and implement, even for beginners, while providing powerful features.

How to Use Socket.IO in Node.js:

1. Install Socket.IO:

You need to install both the server and client libraries:

```
npm install socket.io
```

2. Set Up Socket.IO in Node.js:

Here's how you can set it up on the server side:

```
// Import modules

const express = require('express');

const http = require('http');

const { Server } = require('socket.io');


// Create an Express app

const app = express();

const server = http.createServer(app);


// Initialize Socket.IO

const io = new Server(server);


// Serve a simple index.html file

app.get('/', (req, res) => {

  res.sendFile(__dirname + '/index.html');

});


// Listen for incoming connections

io.on('connection', (socket) => {

  console.log('A user connected');


  // Handle custom events

  socket.on('chat message', (msg) => {

    console.log('Message: ' + msg);


    // Broadcast message to all clients

    io.emit('chat message', msg);
```

```
});
```

```
// Handle disconnection
```

```
socket.on('disconnect', () => {
```

```
  console.log('User disconnected');
```

```
});
```

```
});
```

```
// Start server on port 3000
```

```
server.listen(3000, () => {
```

```
  console.log('Listening on *:3000');
```

```
});
```

3. Client-side Implementation:

On the client side (e.g., in index.html), you need to connect to the server and handle events:

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
    <title>Socket.IO Chat</title>
```

```
  </head>
```

```
  <body>
```

```
    <ul id="messages"></ul>
```

```
    <form id="form" action="">
```

```
      <input id="input" autocomplete="off" /><button>Send</button>
```

```
    </form>
```

```
<!-- Include Socket.IO client library -->
```

```
<script src="/socket.io/socket.io.js"></script>
```

```
<script>

// Connect to the server

var socket = io();


// Send chat message

var form = document.getElementById('form');
var input = document.getElementById('input');


form.addEventListener('submit', function(e) {

  e.preventDefault();

  if (input.value) {

    socket.emit('chat message', input.value);

    input.value = '';

  }

});


// Listen for chat message event

socket.on('chat message', function(msg) {

  var item = document.createElement('li');

  item.textContent = msg;

  document.getElementById('messages').appendChild(item);

});

</script>

</body>

</html>
```

4. Run the Application:

1. Run the Node.js server:

node app.js

2. Visit <http://localhost:3000> in your browser. You should be able to send and receive chat messages in real-time.

Broadcasting Events

1. Server-side Broadcasting

In the server code, you can use the broadcast method to send a message to all connected clients except the one who initiated the event.

Example:

```
io.on('connection', (socket) => {  
  
  console.log('A user connected');  
  
  
  // Listen for a custom event from the client  
  
  socket.on('chat message', (msg) => {  
  
    console.log('Message: ' + msg);  
  
  
    // Broadcast the message to all other clients  
  
    socket.broadcast.emit('chat message', msg);  
  
  });  
  
  
  // Handle disconnection  
  
  socket.on('disconnect', () => {  
  
    console.log('User disconnected');  
  
  });  
  
});
```

In this example:

- When a client sends a message ('chat message'), the server listens to this event.

- The server then uses `socket.broadcast.emit()` to broadcast the message to all clients except the one who sent it.

2. Sending to All Clients Including Sender

If you want to broadcast to **all** clients including the sender, use `io.emit()` instead of `socket.broadcast.emit()`:

```
io.on('connection', (socket) => {
  socket.on('chat message', (msg) => {
    // Emit to all clients including the sender
    io.emit('chat message', msg);
  });
});
```

3. Broadcasting to a Specific Room

Socket.IO allows you to create **rooms**, where clients can join and you can broadcast messages only to members of that room. Here's how you can broadcast to a specific room:

- When a client joins a room:

```
socket.join('room1');
```

- To broadcast a message to everyone in that room (except the sender):

```
socket.to('room1').emit('chat message', msg);
```

This sends the event only to clients connected to 'room1', excluding the sender.

- To send the message to **all clients in the room** including the sender:

```
io.in('room1').emit('chat message', msg);
```

Example of Broadcasting to All Clients Except Sender:

Server-side:

```
io.on('connection', (socket) => {
  console.log('A user connected');

  // When a message is sent by the client
  socket.on('new event', (data) => {
    // Broadcast the message to everyone except the sender
```

```
socket.broadcast.emit('new event', data);  
});
```

```
socket.on('disconnect', () => {  
  console.log('A user disconnected');  
});  
});
```

Client-side:

```
<!DOCTYPE html>  
  
<html>  
  
  <head>  
  
    <title>Socket.IO Broadcasting Example</title>  
  
  </head>  
  
  <body>  
  
    <button id="sendMessage">Send Message</button>  
  
  
    <script src="/socket.io/socket.io.js"></script>  
  
    <script>  
  
      var socket = io();  
  
  
      // Trigger an event when button is clicked  
  
      document.getElementById('sendMessage').addEventListener('click', function() {  
  
        socket.emit('new event', 'This is a broadcast message!');  
  
      });  
  
  
      // Listen for the broadcasted event  
  
      socket.on('new event', function(data) {
```

```
        console.log('Broadcast message received: ' + data);
    });
</script>
</body>
</html>
```

In this case:

- When a user clicks the **"Send Message"** button, a new event is emitted.
- The server broadcasts this message to all clients except the sender using `socket.broadcast.emit()`.
- All other clients receive the broadcasted message.

Summary of Broadcast Methods:

- **`socket.broadcast.emit(event, data)`**: Sends to all connected clients except the sender.
- **`io.emit(event, data)`**: Sends to all connected clients including the sender.
- **`socket.to('room').emit(event, data)`**: Sends to all clients in a specific room, except the sender.
- **`io.in('room').emit(event, data)`**: Sends to all clients in a specific room, including the sender.

Sharing Location:

1. Client-Side: Get Current Location

The **Geolocation API** in the browser allows you to get the user's current position (latitude and longitude). Once you have the location, you can emit it to the server using Socket.IO.

Example of Client-Side Code:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Location Sharing</title>
```

```
</head>

<body>

  <button id="shareLocation">Share My Location</button>

  <ul id="locationList"></ul>


  <!-- Include the Socket.IO client-side library -->

  <script src="/socket.io/socket.io.js"></script>

  <script>

    var socket = io();


    // Get reference to the "Share Location" button

    var shareLocationButton = document.getElementById('shareLocation');

    var locationList = document.getElementById('locationList');


    // Add click event listener to the button

    shareLocationButton.addEventListener('click', function() {

      // Check if Geolocation API is available

      if (!navigator.geolocation) {

        alert('Geolocation is not supported by your browser.');
```

```

    });

    }

  });

  // Listen for location updates from other users
  socket.on('locationUpdate', function(location) {

    var listItem = document.createElement('li');

    listItem.textContent = `User is at Latitude: ${location.latitude}, Longitude: ${location.longitude}`;

    locationList.appendChild(listItem);

  });
</script>
</body>
</html>

```

2. Server-Side: Handle Location Broadcast

Once the client emits the user's current location, the server receives it and can broadcast it to other connected clients.

Example of Server-Side Code:

```

const express = require('express');

const http = require('http');

const { Server } = require('socket.io');

// Create an Express app

const app = express();

const server = http.createServer(app);

const io = new Server(server);

// Serve a simple HTML file for the client
app.get('/', (req, res) => {

```

```
res.sendFile(__dirname + '/index.html');
});

// Handle socket connection
io.on('connection', (socket) => {
  console.log('A user connected');

  // Listen for the "shareLocation" event from the client
  socket.on('shareLocation', (location) => {
    console.log('Received location:', location);

    // Broadcast the location to all other clients except the sender
    socket.broadcast.emit('locationUpdate', location);
  });

  socket.on('disconnect', () => {
    console.log('User disconnected');
  });
});

// Start the server on port 3000
server.listen(3000, () => {
  console.log('Server is listening on *:3000');
});
```

3. Explanation:

- **Client-Side:**
 - When the "Share My Location" button is clicked, the **Geolocation API** retrieves the user's latitude and longitude.

- The location is sent to the server using the `socket.emit()` function with the event 'shareLocation'.
- The client also listens for `locationUpdate` events and displays the locations of other users.
- **Server-Side:**
 - When a client sends their location, the server listens for the 'shareLocation' event.
 - The server then broadcasts the location to all other clients (using `socket.broadcast.emit()`).

This enables real-time location sharing between all connected clients.

4. Enhancements:

- You can modify the `locationList` to display the users' locations on a **map** using libraries like **Google Maps API** or **Leaflet.js** for a better user experience.
- You can also limit location updates to a specific **room** or group of users using Socket.IO's **room** feature if you're dealing with multiple channels.

Capturing Form States and Button Clicks

Let's create a simple HTML form with a submit button. When the form is submitted, the input values are sent to the server, and based on the server response, you can render messages.

Example HTML (Client-side):

html

Copy code

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Form and Button State Example</title>

</head>

<body>

  <form id="userForm">
```

```
<label for="name">Name:</label>
```

```
<input type="text" id="name" name="name" required><br><br>
```

```
<label for="email">Email:</label>
```

```
<input type="email" id="email" name="email" required><br><br>
```

```
<button id="submitBtn" type="submit">Submit</button>
```

```
</form>
```

```
<div id="message"></div>
```

```
<!-- Include the Socket.IO client-side library -->
```

```
<script src="/socket.io/socket.io.js"></script>
```

```
<script>
```

```
var socket = io();
```

```
var form = document.getElementById('userForm');
```

```
var messageDiv = document.getElementById('message');
```

```
var submitBtn = document.getElementById('submitBtn');
```

```
// Listen for form submission
```

```
form.addEventListener('submit', function(e) {
```

```
  e.preventDefault(); // Prevent the form from reloading the page
```

```
  // Capture form data
```

```
  var name = document.getElementById('name').value;
```

```
  var email = document.getElementById('email').value;
```

```

// Disable the submit button after click
submitBtn.disabled = true;

// Send form data to the server using Socket.IO
socket.emit('formSubmit', { name, email });

// Display a message while waiting for the server response
messageDiv.innerHTML = "Submitting...";
});

// Listen for server response after form submission
socket.on('formResponse', function(responseMessage) {
  // Render the response message in the UI
  messageDiv.innerHTML = responseMessage;

  // Re-enable the submit button
  submitBtn.disabled = false;
});
</script>
</body>
</html>

```

2. Server-Side: Handling Form Submission

In Node.js, when the client submits the form, the server receives the form data, processes it, and then responds by sending a message back to the client. The message can be rendered on the client's UI.

Example Node.js Server (using Socket.IO):

```

const express = require('express');

const http = require('http');

```

```
const { Server } = require('socket.io');

// Create an Express app

const app = express();

const server = http.createServer(app);

const io = new Server(server);

// Serve static files (like index.html)

app.get('/', (req, res) => {

  res.sendFile(__dirname + '/index.html');

});

// Handle socket connections

io.on('connection', (socket) => {

  console.log('A user connected');

  // Listen for form submission from the client

  socket.on('formSubmit', (formData) => {

    console.log('Form data received:', formData);

    // Simulate processing the form data (e.g., saving to a database)

    let responseMessage = `Hello ${formData.name}, your email (${formData.email}) has been submitted successfully!`;

    // Send response back to the client

    socket.emit('formResponse', responseMessage);

  });
```

```
// Handle disconnection

socket.on('disconnect', () => {

  console.log('User disconnected');

});

});

// Start the server on port 3000

server.listen(3000, () => {

  console.log('Server is running on http://localhost:3000');

});
```

3. Explanation:

Client-Side:

- **Form Submission:** When the form is submitted, the data from the name and email fields is captured, and the form submission is prevented from refreshing the page.
- **Disabling Button:** The submit button is disabled after being clicked to prevent multiple submissions.
- **Sending Data:** The form data is sent to the server using the `socket.emit('formSubmit', data)` function.
- **Receiving Response:** The client listens for a response from the server on the 'formResponse' event and renders the response message in the `messageDiv`.

Server-Side:

- **Receiving Data:** When the server receives the form data on the 'formSubmit' event, it processes the data and constructs a response message.
- **Sending Response:** The server sends the response message back to the client using `socket.emit('formResponse', message)`.

4. Rendering Messages and Button States:

- On the client-side, you can render different messages depending on the server response.
- You can also enable or disable the form button to control its state (e.g., disabling the submit button while waiting for the server response).

Enhancements:

- **Error Handling:** You can add error messages if the form submission fails or if there is a problem with the server response.
- **Validation:** You can add more validation logic before sending the form data to the server.
- **Form Reset:** After a successful form submission, reset the form fields if needed.

Example Messages Rendering:

If the server processes the form successfully, the client can render a success message:

```
<div id="message">Hello John, your email (john@example.com) has been submitted successfully!</div>
```

If there's an error (e.g., invalid data), you can render an error message:

```
<div id="message" style="color: red;">Error: Invalid email address!</div>
```

This structure provides a simple and efficient way to handle form submissions and render messages in real-time using Node.js and Socket.IO.

Timestamps for determining location of messages:

1. Server-Side: Adding Timestamps

When the server handles incoming messages, it can add a timestamp before broadcasting them to other clients. This ensures that each message is timestamped when it's processed on the server.

Example (Server-Side - Node.js + Socket.IO):

```
const express = require('express');

const http = require('http');

const { Server } = require('socket.io');

const moment = require('moment'); // A library for formatting timestamps

const app = express();

const server = http.createServer(app);

const io = new Server(server);

app.use(express.static('public'));
```

```
io.on('connection', (socket) => {  
  
  console.log('A user connected');  
  
  // Listen for messages from the client  
  socket.on('chatMessage', (messageData) => {  
  
    // Add a timestamp to the message  
    const timestamp = moment().format('YYYY-MM-DD HH:mm:ss'); // Format timestamp  
    const messageWithTimestamp = {  
      message: messageData.message,  
      username: messageData.username,  
      time: timestamp  
    };  
  
    // Send the message with a timestamp to all clients  
    io.emit('messageWithTimestamp', messageWithTimestamp);  
  });  
  
  socket.on('disconnect', () => {  
    console.log('User disconnected');  
  });  
});  
  
// Start the server  
server.listen(3000, () => {  
  console.log('Server is running on http://localhost:3000');  
});
```

In this example:

- We use the **moment.js** library (or you can use the native Date() object) to generate a formatted timestamp whenever a message is received.
- The messageWithTimestamp object contains the original message, the username, and the timestamp.

2. Client-Side: Displaying Messages with Timestamps

When the client receives a message from the server, it can render the message along with the associated timestamp.

Example (Client-Side - HTML + Socket.IO):

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Chat with Timestamps</title>

</head>

<body>

  <div id="messages"></div>

  <form id="messageForm">

    <input id="messageInput" type="text" placeholder="Enter message" required>

    <input id="username" type="text" placeholder="Your name" required>

    <button type="submit">Send</button>

  </form>

  <script src="/socket.io/socket.io.js"></script>

  <script>

    const socket = io();

    const messagesDiv = document.getElementById('messages');

    const form = document.getElementById('messageForm');

    const messageInput = document.getElementById('messageInput');
```

```

const usernameInput = document.getElementById('username');

form.addEventListener('submit', (e) => {

  e.preventDefault();

  const message = messageInput.value;

  const username = usernameInput.value;

  // Send the message and username to the server
  socket.emit('chatMessage', { message, username });

  // Clear the input field after sending
  messageInput.value = "";

});

// Listen for messages from the server with timestamps
socket.on('messageWithTimestamp', (data) => {

  const messageItem = document.createElement('div');

  messageItem.innerHTML = `<strong>${data.username}</strong> [${data.time}]:
  ${data.message}`;

  messagesDiv.appendChild(messageItem);

});

</script>

</body>

</html>

```

In this example:

- When the form is submitted, the message and username are sent to the server.

- The server responds with a message that includes a timestamp, which is then displayed in the messagesDiv.

3. Native JavaScript Date for Timestamps

If you don't want to use external libraries like **moment.js**, you can use native JavaScript to create timestamps:

```
const timestamp = new Date().toLocaleString(); // Gives a timestamp like "9/9/2024, 12:45:36 PM"
```

You can replace the moment() function in the server-side example with the native Date() object.

4. Customizing the Timestamp Format

You can customize how the timestamp is displayed. For example, you can show only the time or both the date and time.

- **For moment.js:**

```
const timestamp = moment().format('HH:mm:ss'); // Only time
```

```
const timestamp = moment().format('DD/MM/YYYY HH:mm:ss'); // Date and time
```

- **For JavaScript Date object:**

```
const timestamp = new Date().toLocaleTimeString(); // Only time
```

```
const timestamp = new Date().toLocaleString(); // Full date and time
```

Summary:

- **Server-Side:** Timestamps are generated when a message is received and processed, then the message with the timestamp is broadcasted to all clients.
- **Client-Side:** Messages with timestamps are displayed in the chat window or message list.

Tracking users joining and leaving:

1. Server-Side: Tracking Connections and Disconnections

You need to listen for connection and disconnect events from **Socket.IO**. When a user joins, you can add them to a list of connected users, and when they leave, you can remove them from that list.

Example (Server-Side - Node.js + Socket.IO):

```
javascript
```

Copy code

```
const express = require('express');
```

```
const http = require('http');

const { Server } = require('socket.io');

const app = express();

const server = http.createServer(app);

const io = new Server(server);

let users = {}; // Object to store connected users with their socket IDs

app.use(express.static('public'));

io.on('connection', (socket) => {

  console.log('A user connected: ', socket.id);

  // When a new user joins, add them to the users object

  socket.on('userJoined', (username) => {

    users[socket.id] = username;

    // Notify all clients about the new user

    io.emit('userUpdate', { users, message: `${username} has joined the chat` });

  });

  // Handle disconnection

  socket.on('disconnect', () => {

    const username = users[socket.id];

    // Remove the user from the users object
```

```

delete users[socket.id];

// Notify all clients about the user leaving
io.emit('userUpdate', { users, message: `${username} has left the chat` });
});

});

// Start the server
server.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});

```

2. Client-Side: Handling User Joining and Leaving

On the client side, you can emit an event when the user joins (e.g., after entering their username), and listen for updates about users joining or leaving.

Example (Client-Side - HTML + Socket.IO):

html

Copy code

```

<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Tracking Users</title>

</head>

<body>

  <div id="chat">

    <div id="users"></div>

    <div id="messages"></div>

```

```
<form id="userForm">

  <input id="usernameInput" type="text" placeholder="Enter your name" required>

  <button type="submit">Join Chat</button>

</form>

</div>
```

```
<!-- Include the Socket.IO client-side library -->

<script src="/socket.io/socket.io.js"></script>

<script>

  const socket = io();

  const usersDiv = document.getElementById('users');

  const messagesDiv = document.getElementById('messages');

  const userForm = document.getElementById('userForm');

  const usernameInput = document.getElementById('usernameInput');

  let username = "";

  // When the user submits their name and joins the chat
  userForm.addEventListener('submit', (e) => {

    e.preventDefault();

    username = usernameInput.value;

    // Hide the form and send the username to the server

    userForm.style.display = 'none';

    socket.emit('userJoined', username);

  });
```

```

// Listen for user updates (join/leave)

socket.on('userUpdate', (data) => {

  // Display the message (user join/leave) in the chat

  const messageItem = document.createElement('div');

  messageItem.innerHTML = data.message;

  messagesDiv.appendChild(messageItem);


  // Update the list of users

  usersDiv.innerHTML = '<h4>Users in chat:</h4>';

  for (let id in data.users) {

    const userItem = document.createElement('div');

    userItem.innerHTML = data.users[id];

    usersDiv.appendChild(userItem);

  }

});

</script>

</body>

</html>

```

3. How It Works:

Server-Side Logic:

1. **Tracking Users:** When a new user joins, the server adds their username to the users object, using their socket.id as the key. The server then broadcasts the updated list of users and a message indicating that the user has joined.
2. **Disconnecting Users:** When a user disconnects, their username is removed from the users object, and a message is sent to all clients indicating that the user has left the chat.

Client-Side Logic:

1. When a user enters their name and submits the form, the username is sent to the server.

2. The client listens for updates from the server (userUpdate), which include the current list of users and any messages about users joining or leaving.
3. The list of users and the messages are displayed dynamically.

4. Example Workflow:

- **User Joins:**
 - The user enters their username in the form and clicks "Join Chat."
 - The client sends the username to the server.
 - The server adds the user to the list of connected users and broadcasts a message like "{username} has joined the chat."
 - All clients update the displayed user list and chat messages.
- **User Leaves:**
 - When the user disconnects (e.g., closes the browser tab), the server removes the user from the list.
 - The server broadcasts a message like "{username} has left the chat."
 - All clients update the displayed user list and chat messages.

5. Improvement Ideas:

- **Persistent Users:** Use a database (e.g., MongoDB) to store connected users if you need to persist data across sessions.
- **Private Chat:** You can extend the logic to allow users to send private messages to each other based on their socket IDs.
- **User Avatars:** Enhance the user experience by adding avatars or additional user info (e.g., profile pictures) when they join the chat.

This structure provides an effective way to track when users join and leave a chatroom or similar application.

Concept of Redis:

Redis is an open-source, in-memory data structure store, often used as a database, cache, and message broker. It supports various data structures, including strings, hashes, lists, sets, and sorted sets. Here are some key concepts and how you can use Redis in a Node.js API:

Concepts of Redis

1. **Data Structures:**

- **Strings:** Simple key-value pairs.
- **Hashes:** Maps between string field and string values, useful for storing objects.
- **Lists:** Collections of ordered elements.
- **Sets:** Collections of unique elements.
- **Sorted Sets:** Collections of unique elements with a score to maintain order.

2. Persistence:

- **RDB (Redis Database Backup):** Snapshots of the database at specified intervals.
- **AOF (Append-Only File):** Logs every write operation received by the server.

3. Replication:

- Master-slave replication allows you to have a primary Redis server and multiple replicas.

4. Pub/Sub:

- Publish/Subscribe messaging paradigm allowing messages to be broadcast to multiple subscribers.

5. Transactions:

- Support for multi-command transactions using commands like MULTI, EXEC, WATCH, and DISCARD.

6. Lua Scripting:

- Allows running Lua scripts to perform complex operations atomically.

Building an API with Redis in Node.js

1. Install Dependencies:

- You'll need redis and express npm packages.

`npm install redis express`

2. Setup Redis Client:

```
const redis = require('redis');
```

```
const client = redis.createClient();
```

```
client.on('error', (err) => {
```

```
  console.error('Redis error: ' + err);
```

```
});
```

3. Create an Express Server:

```
const express = require('express');
```

```
const app = express();
```

```
app.use(express.json());
```

```
// Simple GET endpoint to fetch data from Redis
```

```
app.get('/data/:key', (req, res) => {
```

```
  const { key } = req.params;
```

```
  client.get(key, (err, data) => {
```

```
    if (err) {
```

```
      return res.status(500).json({ error: err.message });
```

```
    }
```

```
    if (data === null) {
```

```
      return res.status(404).json({ message: 'Key not found' });
```

```
    }
```

```
    res.json({ key, value: data });
```

```
  });
```

```
});
```

```
// Simple POST endpoint to store data in Redis
```

```
app.post('/data', (req, res) => {
```

```
  const { key, value } = req.body;
```

```
  client.set(key, value, (err) => {
```

```
    if (err) {
```

```
      return res.status(500).json({ error: err.message });
```

```
    }
```

```
    res.status(201).json({ message: 'Data saved successfully' });  
  });  
});
```

```
// Start the server
```

```
app.listen(3000, () => {  
  console.log('Server is running on port 3000');  
});
```

4. Run the Server:

- Start your Redis server if it's not already running.
- Run your Node.js application:

```
node index.js
```

5. Test the API:

- Use tools like curl, Postman, or any HTTP client to test your API endpoints.

Store data

```
curl -X POST http://localhost:3000/data -H "Content-Type: application/json" -d '{"key": "test",  
"value": "123"}'
```

Fetch data

```
curl http://localhost:3000/data/test
```

Chat Application:

Project Structure

Here's an example of a project structure that includes all the functionalities:

```
bash
```

Copy code

```
/my-chat-app
```

```
  /public
```

index.html

style.css

client.js

server.js

package.json

Step-by-Step Guide

1. Project Setup

1. Create the Project Directory:

```
mkdir my-chat-app
```

```
cd my-chat-app
```

2. Initialize a Node.js Project:

```
npm init -y
```

3. Install Dependencies:

```
npm install express socket.io moment
```

2. Server-Side Code

Create server.js in the root directory to handle the server-side logic.

server.js

```
const express = require('express');
```

```
const http = require('http');
```

```
const { Server } = require('socket.io');
```

```
const moment = require('moment'); // For handling timestamps
```

```
const app = express();
```

```
const server = http.createServer(app);
```

```
const io = new Server(server);
```

```
let users = {}; // Object to store connected users with their socket IDs
```

```
app.use(express.static('public'));
```

```
io.on('connection', (socket) => {
```

```
  console.log('A user connected: ', socket.id);
```

```
  // When a new user joins
```

```
  socket.on('userJoined', (username) => {
```

```
    users[socket.id] = username;
```

```
    const timestamp = moment().format('YYYY-MM-DD HH:mm:ss');
```

```
    io.emit('userUpdate', { users, message: `${username} has joined the chat`, time: timestamp });
```

```
  });
```

```
  // Handle message with timestamp
```

```
  socket.on('chatMessage', (messageData) => {
```

```
    const timestamp = moment().format('YYYY-MM-DD HH:mm:ss');
```

```
    const messageWithTimestamp = {
```

```
      message: messageData.message,
```

```
      username: messageData.username,
```

```
      time: timestamp
```

```
    };
```

```
    io.emit('messageWithTimestamp', messageWithTimestamp);
```

```
  });
```

```
  // Handle disconnection
```

```
  socket.on('disconnect', () => {
```

```
    const username = users[socket.id];
```

```
delete users[socket.id];

const timestamp = moment().format('YYYY-MM-DD HH:mm:ss');

io.emit('userUpdate', { users, message: `${username} has left the chat`, time: timestamp });

});

});
```

```
// Start the server

server.listen(3000, () => {

  console.log('Server is running on http://localhost:3000');

});
```

3. Client-Side Code

Create the following files in the public directory:

1. index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Chat with Socket.IO</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

  <div id="chat">

    <div id="users"></div>

    <div id="messages"></div>

    <form id="userForm">

      <input id="usernameInput" type="text" placeholder="Enter your name" required>
```

```
<button type="submit">Join Chat</button>

</form>

<form id="messageForm" style="display: none;">

  <input id="messageInput" type="text" placeholder="Enter your message" required>

  <button type="submit">Send</button>

</form>

</div>
```

```
<!-- Include the Socket.IO client-side library -->

<script src="/socket.io/socket.io.js"></script>

<script src="client.js"></script>

</body>

</html>
```

2. **style.css**

```
#chat {

  width: 500px;

  margin: 0 auto;

}

#messages {

  border: 1px solid #ccc;

  height: 300px;

  overflow-y: scroll;

  margin-bottom: 10px;

}
```

3. **client.js**

```
const socket = io();
```

```
const usersDiv = document.getElementById('users');
const messagesDiv = document.getElementById('messages');
const userForm = document.getElementById('userForm');
const messageForm = document.getElementById('messageForm');
const usernameInput = document.getElementById('usernameInput');
const messageInput = document.getElementById('messageInput');
```

```
let username = '';
```

```
// When the user submits their name and joins the chat
```

```
userForm.addEventListener('submit', (e) => {
  e.preventDefault();
```

```
  username = usernameInput.value;
```

```
  userForm.style.display = 'none';
```

```
  messageForm.style.display = 'block';
```

```
  socket.emit('userJoined', username);
```

```
});
```

```
// When the user submits a message
```

```
messageForm.addEventListener('submit', (e) => {
```

```
  e.preventDefault();
```

```
  const message = messageInput.value;
```

```
  socket.emit('chatMessage', { message, username });
```

```
  messageInput.value = '';
```

```
});
```

```

// Listen for user updates (join/leave)

socket.on('userUpdate', (data) => {

  const messageItem = document.createElement('div');

  messageItem.innerHTML = `[${data.time}] ${data.message}`;

  messagesDiv.appendChild(messageItem);


  // Update the list of users

  usersDiv.innerHTML = '<h4>Users in chat:</h4>';

  for (let id in data.users) {

    const userItem = document.createElement('div');

    userItem.innerHTML = data.users[id];

    usersDiv.appendChild(userItem);

  }

});


// Listen for messages with timestamps

socket.on('messageWithTimestamp', (data) => {

  const messageItem = document.createElement('div');

  messageItem.innerHTML = `<strong>${data.username}</strong> [${data.time}]: ${data.message}`;

  messagesDiv.appendChild(messageItem);

});

```

4. Running the Application

1. Start the Server:

```
node server.js
```

2. Open a Browser:

Visit <http://localhost:3000> to see your chat application. Open multiple tabs or different browsers to test user connections, disconnections, and message timestamps.

Summary of the Combined Project:

1. Server-Side (server.js):

- Handles user connections, disconnections, and broadcasting messages with timestamps.
- Uses **Socket.IO** for real-time communication and **moment.js** for formatting timestamps.

2. Client-Side (public/index.html, public/style.css, public/client.js):

- Provides a form for users to join the chat and send messages.
- Displays messages and user updates with timestamps.
- Updates the user list dynamically as users join or leave.